💻 Evidencia 2

TECMILENIO

Carlos David Duarte Gill

Maestría en Gestión de Tecnologías de la Información

Gestión de calidad de datos

- Alumno: Carlos David Duarte Gill
- Matricula: 7089642
- Correo: AL07089642@tecmilenio.mx
- Profesor: Heriberto Gonzalez Cazares
- Fecha: lunes 9 de marzo de 2026

Cursando la Maestría en Gestión de Tecnologías de la Información

📘 Introducción – Evidencia 2

La Evidencia 2 representa el final del curso **Gestión de Calidad de Datos**, donde se revisaron conocimientos sobre la limpieza técnica para establecer una fuente de datos confiable. En esta segunda etapa, el proyecto evolucionará de solo un script de validación aislado a un pipeline capaz de integrar diversas fuentes de información y mantener una postura proactiva ante la posibilidad de recibir datos con baja calidad.

🗽 Alcance y propósito de la Evidencia 2

El alcance de la evidencia 2 integra los siguientes puntos:

- Integración de nuevas fuentes de datos para demostrar la capacidad de orquestación y transformación cruzada de datos, y, para casos de estudio nuestras fuentes de datos estarán simuladas.
- Implementación de tableros de control y distintas métricas que permitan visualizar la salud de los datos en tiempo real.
- Documentación de la trazabilidad y procedencia de los datos, asegurando el cumplimiento de principios éticos y legales de privacidad.
- Diseño de un plan de respuesta ante incidentes de calidad de datos.
- Creación de reportes de alta fidelidad orientados a la toma de decisiones..

💻 Desarrollo de la Evidencia 2

🤖 Entorno Python

El entorno de trabajo para la Evidencia 2 se bajo en el entorno de trabajo para la Evidencia 1, el cuál esta optimizado para el análisis de datos masivos y la automatización de procesos de calidad mediante `Python 3.10` y herramientas de limpieza avanzada como `ydata-profiling` y `yjanitor`.

A continuación, describo los componentes clave del entorno funcional:

Especificaciones del Entorno para el análisis de datos:

- Núcleo de ejecución sobre Python 3.10.
- Gestión dual de paquetes mediante `Conda` y `Pip`, para instalar binarios pesados mediante **conda** como `ydata-profiling` y **pip** para librerías de manipulación como `pandas`, se aseguran que las dependencias compartidas se resuelvan sin conflictos de versiones.

Librerías principales:

1. **Pandas**: Como motor principal del proyecto.
2. **PyJanitor**: Esta extensión de Pandas actúa como una capa de "higiene de datos".
3. **YData Profiling**: Es una herramienta de diagnóstico automatizado.
4. **Great Expectations**: Es una librería para validación de datos, pruebas y documentación.

🔧 Pasos

1. Crear enviroment

```
conda create -n janitor-p10 python=3.10
```

2. Activar enviroment

```
conda activate janitor-p10
```

3. Dependencias pip

```
pip install ydata-profiling pyjanitor pandas
```

```
pip install great-expectations
```

4. Dependencias conda

```
conda install -c conda-forge ydata-profiling pyjanitor
```

⚙️ Versiones instaladas

Herramienta	Versión
Python	✅ 3.10.19
Pandas	✅ 2.3.3
NumPy	✅ 2.2.6
PyJanitor	✅ 0.32.5
YData Profiling	✅ 0.0.dev0
Great Expectations	✅ 1.14.0

🏠 Selección del conjunto de datos

Para cumplir con los requisitos de la Evidencia 2 y asegurar todos los pasos requeridos durante su desarrollo, he decidido utilizar el conjunto de datos de **fashion_products.csv** propuesto en Canvas de Tecmilenio.

📒 Acerca del conjunto de datos

El archivo `fashion_products.csv` parece ser un conjunto de datos se encuentra en un estado de calidad óptima.

Resumen del nuevo Dataset

- Registros: 1,000 filas.
- Columnas: 9 variables (Mezcla de categóricas y numéricas).
- Temática: Comercio electrónico de productos de moda (Adidas, Gucci, Nike, etc.).

Diagnóstico por Dimensiones

Dimensión	Estado	Hallazgo Técnico
Completitud	🟢 Excelente	0% de valores nulos.
Unicidad	🟢 Excelente	0 registros duplicados exactos.
Consistencia	🟢 Alta	Las variables categóricas como Brand, Color, Size y Category están bien estandarizadas sin errores de escritura.
Validez	🟢 Alta	Los precios de los productos oscilan entre $10 y 100$ y los ratings entre 1.0 y 5.0, cumpliendo con los rangos lógicos de negocio.
Precisión	🟢 Alta	No se detectan valores atípicos extremos.

🔗 Carga el conjunto de datos en url pública

Se utilizarán las url's públicas de los Dataset descritas en Canvas de Tecmilenio:

- **Fuente 1:** product_sales_dataset_final.csv
- **Fuente 2:** fashion_products.csv

🩺 Revisión del pipeline existente

Del notebook anterior, *Evidencia_1_Carlos_Duarte*, se destacan los siguientes puntos que ya se encuentran bajo control:

1. Configuración exitosa de un ambiente con Python 3.10, Pandas 2.x, PyJanitor y Great Expectations 1.x.
2. Identificación clara de inconsistencias en encabezados y formatos de fecha.
3. Limpieza efectiva mediante el encadenamiento de métodos de Janitor como clean_names y remove_empty.
4. Implementación de un suite de expectativas funcional que garantiza unicidad y completitud.

🧬 Desarrollo del nuevo pipeline

A continuación:

🧰 Se importan las librerías necesarias para el análisis y la visualización como Pandas Profiling, PyJanitor, entre otras.

🔧 Se carga el dataset y se muestra una tabla descriptiva del dataset.

🧰 Carga de todas las librerías necesarias

In [10]:
```
# ----------------------------------------------------------------------------
# LIBRERIAS
# ----------------------------------------------------------------------------
import sys
import os
# warnings
import warnings
warnings.filterwarnings('ignore')

# Prueba rápida de carga
try:
    print("⚙️ Cargando librerías de principales...")
    print("-" * 80)
    import pandas as pd
    import numpy as np
    import janitor  # PyJanitor
    import ydata_profiling
    from ydata_profiling import ProfileReport
    import great_expectations as ge
```

```python
    print(f"Python: {sys.version.split()[0]}")
    print(f"Pandas version: {pd.__version__}")
    print(f"NumPy version: {np.__version__}")
    print(f"PyJanitor version: {janitor.__version__}")
    print(f"YData Profiling version: {ydata_profiling.__version__}")
    print(f"Great Expectations version: {ge.__version__}")

except Exception as e:
    print(f"❌ Error de entorno, favor de instalar las dependencias necesarias: {e}")

import matplotlib.pyplot as plt
import seaborn as sns
warnings.filterwarnings('ignore')

print("-" * 80)
print("⚙️ Cargando librerías de visualización...")
print(f"Matplotlib - version: {plt.matplotlib.__version__}")
print(f"Seaborn version: {sns.__version__}")

print("-" * 80)
print("🚀 ¡Entorno preparado!")
print("-" * 80)
```

```
⚙️ Cargando librerías de principales...
--------------------------------------------------------------------------------
Python: 3.10.19
Pandas version: 2.3.3
NumPy version: 2.2.6
PyJanitor version: 0.32.5
YData Profiling version: 0.0.dev0
Great Expectations version: 1.14.0
--------------------------------------------------------------------------------
⚙️ Cargando librerías de visualización...
Matplotlib - version: 3.10.0
Seaborn version: 0.13.2
--------------------------------------------------------------------------------
🚀 ¡Entorno preparado!
--------------------------------------------------------------------------------
```

🗂️ Funciones auxiliares

In [2]:

```python
# ---------------------------------------------------------------------------
# 🖨️ display mensaje
# ---------------------------------------------------------------------------
def display_mensaje(msj):
    print("\n" + "="*80)
```

```
    print("\033[1m --- " + msj + " --- \033[0m")
    print("="*80)
```

🐌 Integración y transformación de nuevos datos

A continuación se inicia el primer paso del nuevo pipeline y se documenta formalmente la nueva fuente de datos, enseguida se aplica un script de transformación que estandarice la estructura de la segunda fuente de datos para que sea compatible con el pipeline construido en la Evidencia 1.

📃 Documentación de la nueva fuente de datos

Como parte de las buenas prácticas de Gobierno de Datos, todo nuevo activo debe ser catalogado antes de ingresar al Data Lake o Pipeline analítico.

- Nombre del activo: fashion_products.csv
- Origen: Datos simulados desde una fuente externa y estructurada accesible mediante url pública de Canvas en Tecmilenio.
- Formato: Archivo CSV.
- Nivel de detalle: Cada fila representa la interacción de un usuario con un producto de moda específico.
- Diccionario de variables original:
  - User ID: Identificador numérico único del cliente/usuario.
  - Product ID: Identificador numérico único del artículo.
  - Product Name: Nombre genérico del artículo, en texto.
  - Brand: Marca fabricante, en texto.
  - Category: Segmento de mercado, en texto.
  - Price: Precio de lista del artículo, decimal.
  - Rating: Calificación otorgada al producto, decimal.
  - Color: Variante de color del producto, en texto.
  - Size: Variante de talla, en texto.

⤴️ Estrategia de integración y transformación

Para que el nuevo dataset pueda coexistir con el dataset original, *product_sales_cleaned.csv*, se aplicaran las siguientes reglas de homologación:

1. Estandarización de nombres de columnas mediante el uso de `clean_names()` de PyJanitor para pasar todos los nombres al formato `snake_case` igualando el estándar de la Evidencia 1.
2. Homologación de nombres de "columnas parecidas":
   - Se renombrará la columna `price` a `unit_price` para que el script de analisís entienda que hablamos de la misma métrica financiera.
   - Se renombrará la columna `user_id` a `customer_id` para alinearlo con el concepto de `customer_name` del pipeline original.
3. Alineación de tipos de datos en todas las columnas, por ejemplo, en `unit_price` se deberá asegurar que sea de tipo *float* para permitir validaciones matemáticas en el nuevo pipeline.

♻️ Transformación del segundo dataset

Script de transformación e integración de la nueva fuente de datos.

In [12]:
```
# ---------------------------------------------------------------------------
# Transformación e integración de nueva fuente de datos
# ---------------------------------------------------------------------------

display_mensaje("INICIANDO INTEGRACIÓN DE NUEVA FUENTE")

# nueva fuente de datos
_URL_2_ = "https://utm-cdn-labcontenidos-htfaarehf2gcfycs.a01.azurefd.net/contenido/maestria/mtca2405/anexos/actividades/v3/fashion_products.csv"
# _URL_2_ = "./fashion_products.csv"
# config
pd.set_option('display.max_columns', None)

df_fashion = pd.read_csv(_URL_2_)

# transformación del dataset para homologar con el pipeline de la evidencia 1
df_fashion_integrated = (
    df_fashion
    # formato y nombres de columnas
    .clean_names()
    .rename_column('price', 'unit_price')
    .rename_column('user_id', 'customer_id')
)

# tipos de datos
# decimales
df_fashion_integrated['unit_price'] = df_fashion_integrated['unit_price'].astype(float)
# ID's en formato string para evitar problemas de integraciones
df_fashion_integrated['customer_id'] = df_fashion_integrated['customer_id'].astype(str)
df_fashion_integrated['product_id'] = df_fashion_integrated['product_id'].astype(str)

# validación de nueva estructura
print("\n✅ Estructura del dataset integrado:")
print(df_fashion_integrated.info())

print("\n✅ Muestra de datos homologados:")
display(df_fashion_integrated.head(3))

# guardamos nuevo dataset estandarizado
_DATASET_INTEGRATED_ = "fashion_products_integrated.csv"
```

```python
df_fashion_integrated.to_csv(_DATASET_INTEGRATED_, index=False)
print(f"\n💾 Dataset guardado exitosamente como '{_DATASET_INTEGRATED_}'")
```

```
================================================================================
 --- INICIANDO INTEGRACIÓN DE NUEVA FUENTE ---
================================================================================

✅ Estructura del dataset integrado:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   customer_id   1000 non-null   object
 1   product_id    1000 non-null   object
 2   product_name  1000 non-null   object
 3   brand         1000 non-null   object
 4   category      1000 non-null   object
 5   unit_price    1000 non-null   float64
 6   rating        1000 non-null   float64
 7   color         1000 non-null   object
 8   size          1000 non-null   object
dtypes: float64(2), object(7)
memory usage: 70.4+ KB
None

✅ Muestra de datos homologados:
```

	customer_id	product_id	product_name	brand	category	unit_price	rating	color	size
0	19	1	Dress	Adidas	Men's Fashion	40.0	1.043159	Black	XL
1	97	2	Shoes	H&M	Women's Fashion	82.0	4.026416	Black	L
2	25	3	Dress	Adidas	Women's Fashion	44.0	3.337938	Yellow	XL

```
💾 Dataset guardado exitosamente como 'fashion_products_integrated.csv'
```

🤖 Ampliación del pipeline original

Para esta fase de la Evidencia Final, se procede a consolidar ambas fuentes de datos:

- product_sales_cleaned.csv
- fashion_products_integrated.csv

Con el objetivo de crear un Dataset Enriquecido y enseguida someterlo a un nuevo análisis de calidad automatizado.

👉 Integración de los Datasets

Dado que las fuentes de datos tienen orígenes distintos, se utilizará Pandas para extraer el producto en el **catálogo de ventas** y así poder cruzarlo con el **catálogo de moda**. De esta manerá podremos asegurar que los scripts hablen del mismo producto y que se puedan unir las tablas correctamente.

Dadas diferentes iteraciones en la integración, se ha identificado un patrón estructural en el nombre del producto, po rlo cual, se asume que que la estructura del nombre es $Marca$Producto, de esta manera se establecerá que solo la última palabra del product_name corresponde al tipo de artículo, por ejemplo "Jeans" o "Boots" y todo lo anterior corresponde a la marca o modelo, por ejemplo "Levi's" u "Old Navy".

In [4]:
```
# ------------------------------------------------------------------------
# Integración de los Datasets
# ------------------------------------------------------------------------
display_mensaje("INICIANDO FASE DE INTEGRACIÓN DE DATOS")

# cargando datasets limpios y procesados
df_sales = pd.read_csv('product_sales_cleaned.csv')
df_fashion = pd.read_csv('fashion_products_integrated.csv')

# ------------------------------------------------------------------------
# extracción avanzada de marca y nombre del producto en las ventas
# función para extraer la marca
def extract_brand(name):
    parts = str(name).split()
    if len(parts) > 1:
        return " ".join(parts[:-1])
    return "Unknown"

# ------------------------------------------------------------------------
# función para extraer el tipo de producto desde las ventas
def extract_product(name):
    return str(name).split()[-1].capitalize()
```

```python
# nuevas variables para facilitar la integración
df_sales['extracted_brand'] = df_sales['product_name'].apply(extract_brand)
df_sales['product_base_name'] = df_sales['product_name'].apply(extract_product)

# estandarizando el catálogo de moda
df_fashion['product_base_name'] = df_fashion['product_name'].str.capitalize()

# ----------------------------------------------------------------------
# consolidando el catálogo de moda
'''
# Para evitar duplicar ventas, se crea un catálogo único por producto base,
# promediando el rating y tomando un color/talla representativo.
'''
fashion_catalog = df_fashion.groupby('product_base_name').agg({
    'rating': 'mean',
    'color': lambda x: x.mode()[0] if not x.mode().empty else 'Standard',
    'size': lambda x: x.mode()[0] if not x.mode().empty else 'M'
}).reset_index()

# integración estructural mediante left join
df_integrated = pd.merge(
    df_sales,
    fashion_catalog,
    on='product_base_name',
    how='left'
)

# asegurando el nombre de la marca utilizamos la marca extraída de las ventas de forma nativa.
df_integrated['brand'] = df_integrated['extracted_brand']


# ----------------------------------------------------------------------
# revision de resultados de la integración
display_mensaje("Revisando la estructura del dataset post-integración")
# print(df_integrated.info())


# manejo de nulos después de la integración
# si el producto base no estaba en el catálogo, se le asignan valores por defecto
df_integrated['rating'] = df_integrated['rating'].fillna(df_integrated['rating'].median())
df_integrated['color'] = df_integrated['color'].fillna('Standard')
df_integrated['size'] = df_integrated['size'].fillna('M')

# limpiando las columnas auxiliares
df_integrated = df_integrated.drop(columns=['extracted_brand', 'product_base_name'])

# guardando el dataset final integrado
df_integrated.to_csv('master_sales_integrated.csv', index=False)
```

```
# ----------------------------------------------------------------------
display_mensaje("Integración exitosa")
print ("\n✅ Dataset final guardado como 'master_sales_integrated.csv'")
print(f"Total de registros: {len(df_integrated)}")
print("Nuevas columnas añadidas: brand, rating, color, size")
```

```
================================================================================
 --- INICIANDO FASE DE INTEGRACIÓN DE DATOS ---
================================================================================

================================================================================
 --- Revisando la estructura del dataset post-integración ---
================================================================================

================================================================================
 --- Integración exitosa ---
================================================================================

✅ Dataset final guardado como 'master_sales_integrated.csv'
Total de registros: 200000
Nuevas columnas añadidas: brand, rating, color, size
```

Mejoras añadidas al dataset final

- Con las mejoras añadidas ahora en el registro de venta de cualquier producto se especificará la marca correcta.
- Al usar un *groupby* durante el merge se aseguró de que cada venta del dataset original siga siendo exactamente una fila, manteniendo los 200,000 registros iniciales.
- El nuevo dataset ahora incluye características reales de la venta, como la marca exacta, combinadas con atributos enriquecidos del catálogo, como el rating de cada producto.

✔️ Validaciones al dataset integrado con Great Expectations

En esta sección se establecen **puntos de control** para validar las nuevas variables y la interacción entre ellas sobre el nuevo dataset integrado.

In [ ]:
```
# ----------------------------------------------------------------------
# Validaciones con Great Expectations
# ----------------------------------------------------------------------
"""
Entorno:
- Python: 3.10.19
- pandas: 2.3.3
- numpy: 2.2.6
- great-expectations: 1.14.0

** Código de validación de calidad de datos para el dataset integrado **
```

```python
** El script generado en la Evidencia 1 se adapta para validar el nuevo dataset integrado, con nuevas expectativas específicas para las columnas añadidas. **
"""

# librerías para validación
import os
from great_expectations.checkpoint import Checkpoint
from great_expectations.core.batch_definition import BatchDefinition
from great_expectations.core.validation_definition import ValidationDefinition
from great_expectations.expectations import (
    ExpectColumnValuesToBeBetween,
    ExpectColumnValuesToBeInSet,
    ExpectColumnValuesToNotBeNull
)

display_mensaje("INICIANDO VALIDACIÓN DEL DATASET INTEGRADO")

# ---------------------------------------------------------------------
# configuración del contexto y asset de datos
# ---------------------------------------------------------------------
context = ge.get_context()

# asset de aatos con el archivo limpio
datasource_name = "master_datasource"
datasource = context.data_sources.add_pandas(name=datasource_name)

asset_name = "master_integrated_asset"
asset = datasource.add_csv_asset(
    name=asset_name,
    filepath_or_buffer="master_sales_integrated.csv"
)

batch_definition = asset.add_batch_definition(name="default")
batch = batch_definition.get_batch()
print(batch.batch_definition)

# ---------------------------------------------------------------------
# Suite + Validator
# ---------------------------------------------------------------------

# suite de expectativas
suite_name = "integrated_quality_suite"
suite = context.suites.add(ge.ExpectationSuite(name=suite_name))

# batchrequest
batch_request = asset.build_batch_request()
print(batch_request)
```

```python
# validator (se une batch request + suite)
validator = context.get_validator(
    batch_request=batch_request,
    expectation_suite=suite,
)

# cols
print("Columnas detectadas:", validator.get_expectation_suite)

# ---------------------------------------------------------------------
# Expectativas
# ---------------------------------------------------------------------

# a) expectativas de rango de valores validos para el rating
# el rating del producto debe estar solamente entre 1.0 y 5.0
suite.add_expectation(ExpectColumnValuesToBeBetween(
    column="rating",
    min_value=1.0,
    max_value=5.0
))

# b) expectativas de categorías válidas para la talla
# tallas válidas: XS, S, M, L, XL, XXL
tallas_validas = ["XS", "S", "M", "L", "XL", "XXL"]
suite.add_expectation(ExpectColumnValuesToBeInSet(
    column="size",
    value_set=tallas_validas
))

# c) expectativas de cero valores nulos en la marca
# se verifica que no existan marcas nulas
suite.add_expectation(ExpectColumnValuesToNotBeNull(column="brand"))

# ---------------------------------------------------------------------
# Validaciones
# ---------------------------------------------------------------------

# registro de ValidationDefinition y Checkpoint para ejecutar las validaciones
validation_def_name = "val_def_master_integrated"
validation_definition = context.validation_definitions.add(
    ValidationDefinition(
        name=validation_def_name,
        data=batch_definition,
        suite=suite,
    )
)

# registro del Checkpoint
```

```python
checkpoint_name = "master_checkpoint"
checkpoint = context.checkpoints.add(
    Checkpoint(
        name=checkpoint_name,
        validation_definitions=[validation_definition],
    )
)

# ---------------------------------------------------------------------
# Ejecucion
# ---------------------------------------------------------------------
print("Ejecutando validaciones...")
result = checkpoint.run()

# generación de reportes
context.build_data_docs()

for site in context.get_docs_sites_urls() or []:
    print(f"📄 {site.get('site_name')}: {site.get('site_url')}")


print(f"\n✅ Validación de integración completada. Éxito global: {result.success}")
print(f"📁 Reporte HTML generado. Revisa los Data Docs para ver el detalle de los nuevos cruces.")
```

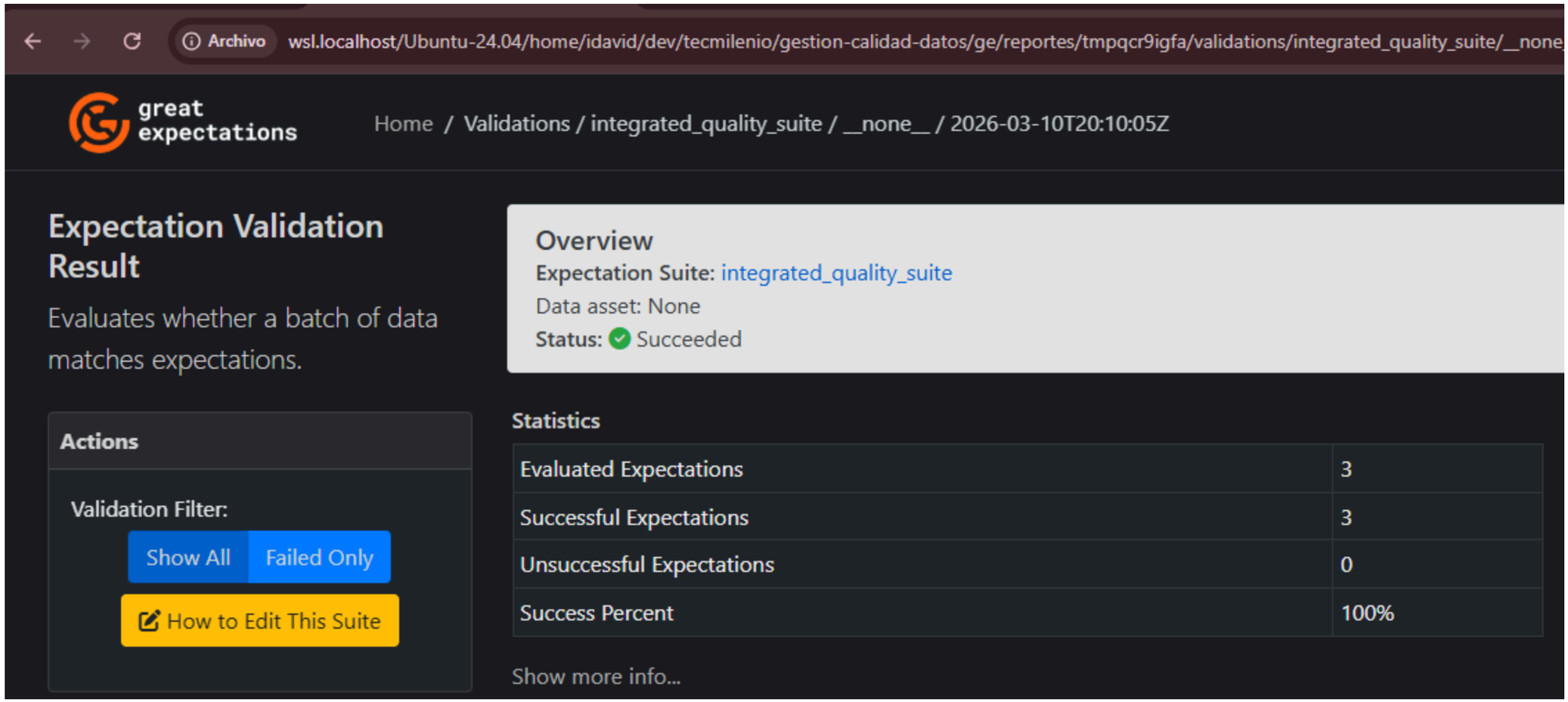
```
================================================================================
 --- INICIANDO VALIDACIÓN DEL DATASET INTEGRADO ---
================================================================================
{
  "datasource_name": "master_datasource",
  "data_connector_name": "fluent",
  "data_asset_name": "master_integrated_asset",
  "batch_identifiers": {}
}
datasource_name='master_datasource' data_asset_name='master_integrated_asset' options={} partitioner=None
Columnas detectadas: <bound method Validator.get_expectation_suite of <great_expectations.validator.validator.Validator object at 0x7fa066941600>>
Ejecutando validaciones...
Calculating Metrics:   0%|          | 0/25 [00:00<?, ?it/s]
📄 local_site: file:///tmp/tmpp2vo3ofn/index.html

✅ Validación de integración completada. Éxito global: True
📁 Reporte HTML generado. Revisa los Data Docs para ver el detalle de los nuevos cruces.
```

Los resultados del pipeline actual demuestran como el script automatizado actual puede enriquecer datos al cruzar las ventas con el catálogo de moda, se ha creado un recurso de datos más valioso para detectar patrones de ventas e ingresos. Todo esto mediante la implementación de los datos integrados y las validaciones automatizadas de Great Expectations en el pipeline.

La herramienta Great Expectations es sumamente valiosa en este punto del pipeline ya que garantiza que cualquier modelo predictivo que use este dataset no sufrirá errores matemáticos por errores en el formato o estructura del dataset.

Documento de validaciones

Se adjunta documento de validaciones generado por Great Expectations.



📐 Implementación de métricas e indicadores de calidad

En esta sección se definen, calculan y visualizan las métricas clave de calidad de datos para el nuevo dataset integrado. En este paso se mejorá el pipeline para que tenga la capacidad de evaluar la calidad de los datos de manera constante cada vez que sea ejecutado.

✔️ Definición de las métricas de calidad

Lista de métricas definidas a partir de los indicadores clave:

1. **Completitud**: Se mide la ausencia de valores nulos en el dataset integrado.
2. **Unicidad**: Evalúa que no existan transacciones duplicadas tomando como base la llave primaria del dataset integrado.
3. **Consistencia**: Verifica que la lógica de negocio se cumpla, para el caso de estudio se evalua: $Revenue = Quantity \times Unit_Price$.
4. **Precisión**: Mide que los valores estén dentro de rangos reales y válidos, por ejemplo que el *rating* esté entre 1.0 y 5.0, y que los precios sean mayores a cero.
5. **Puntualidad**: Evalúa que las fechas de las órdenes sean coherentes temporalmente, por ejemplo, que no existan fechas nulas o fuera de tiempo.

🎖️ Implementación de métricas de calidad

In [6]:
```
# ------------------------------------------------------------------------
#  🎖️ Implementación de métricas de calidad
# ------------------------------------------------------------------------

display_mensaje("CALCULANDO MÉTRICAS DE CALIDAD DE DATOS")

# dataset integrado
df_metricas = pd.read_csv('master_sales_integrated.csv')
df_metricas['order_date'] = pd.to_datetime(df_metricas['order_date'])

total_rows = len(df_metricas)
print(f"Total de filas cargadas: {total_rows}")

# ------------------------------------------------------------------------
# calculando métricas de calidad

# 1. Completitud: % de celdas no nulas en todo el dataset
total_cells = total_rows * df_metricas.shape[1]
missing_cells = df_metricas.isnull().sum().sum()
_COMPLETITUD_ = ((total_cells - missing_cells) / total_cells) * 100

# 2. Unicidad: % de Order_IDs únicos
_UNICICIDAD_ = (df_metricas['order_id'].nunique() / total_rows) * 100
```

```python
# 3. Consistencia: % de filas donde Revenue == Quantity * _unit_price_ (considerando un margen de error por decimales)
df_metricas['calculated_revenue'] = df_metricas['quantity'] * df_metricas['_unit_price_']
filas_consistentes = (abs(df_metricas['_revenue_'] - df_metricas['calculated_revenue']) < 0.01).sum()
_CONSISTENCIA_ = (filas_consistentes / total_rows) * 100

# 4. Precisión: % de filas con Rating válido y Precios válidos
filas_precisas = ((df_metricas['rating'] >= 1.0) & (df_metricas['rating'] <= 5.0) & (df_metricas['_unit_price_'] > 0)).sum()
_PRECISION_ = (filas_precisas / total_rows) * 100

# 5. Puntualidad: % de fechas válidas
# se define la fecha de corte como el 31 de diciembre de 2024
fecha_corte = pd.to_datetime('2024-12-31')
filas_puntuales = (df_metricas['order_date'] <= fecha_corte).sum()
_PUNTUALIDAD_ = (filas_puntuales / total_rows) * 100

# resultados de las metricas para reportes
resultados_metricas = pd.DataFrame({
    'Métrica': ['Completitud', 'Unicidad', 'Consistencia', 'Precisión', 'Puntualidad'],
    'Porcentaje (%)': [_COMPLETITUD_, _UNICICIDAD_, _CONSISTENCIA_, _PRECISION_, _PUNTUALIDAD_]
})

print("\n📊 Tabla Resumen de Métricas:")
display(resultados_metricas.round(2))
```

```
================================================================================
 --- CALCULANDO MÉTRICAS DE CALIDAD DE DATOS ---
================================================================================
Total de filas cargadas: 200000

📊 Tabla Resumen de Métricas:
```

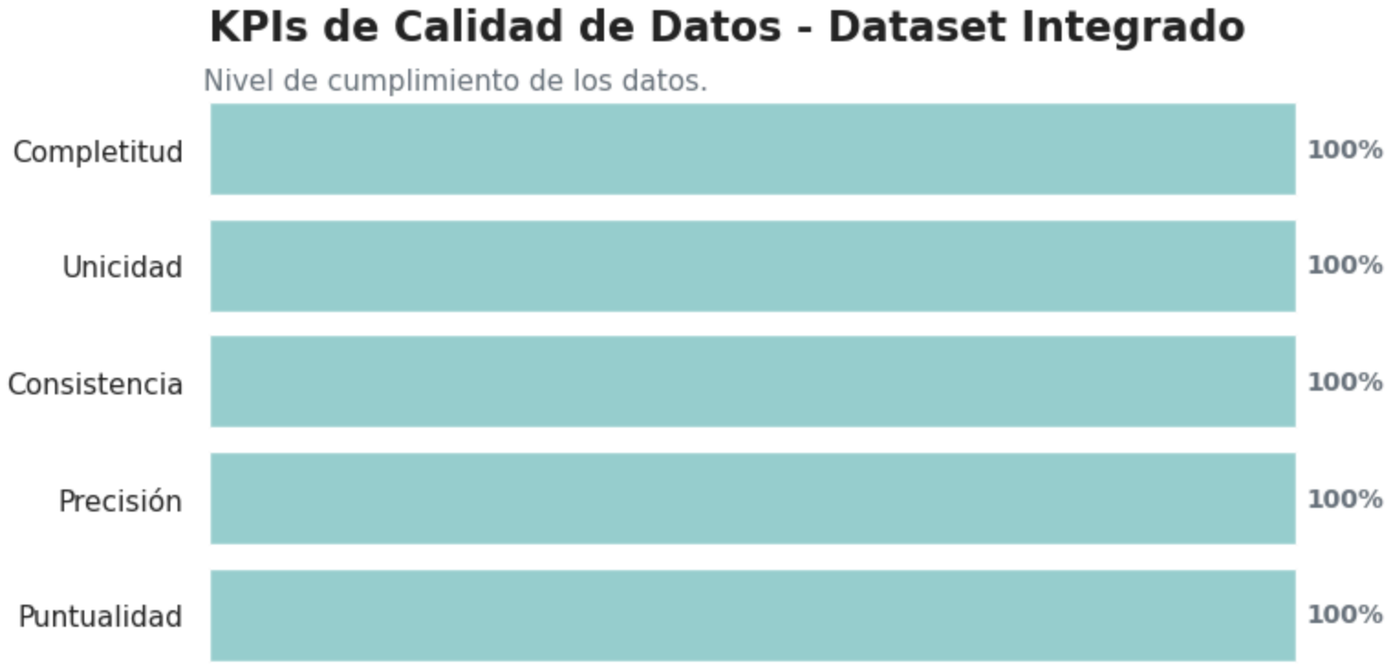
	Métrica	Porcentaje (%)
0	Completitud	100.0
1	Unicidad	100.0
2	Consistencia	100.0
3	Precisión	100.0
4	Puntualidad	100.0

📊 KPIs de calidad de datos en el dataset integrado

In [16]:
```
# ------------------------------------------------------------------
# Configuración del estilo del gráfico
# ------------------------------------------------------------------
sns.set_theme(style="whitegrid")
sns.set_style("white")
warnings.filterwarnings('ignore')

# layout limpio
plt.style.use('tableau-colorblind10')

# colores
_color_primary="#007bff"
_color_danger="#ff0000"
_color_success="#28a745"
_color_tecmi_light="#26d07c"
_Light_Blue_ = '#8fd7d7'
_color_gray="#6c757d"
_color_white="#ffffff"
_Gray_ = '#c8c8c8'
```

In [8]:
```
# ------------------------------------------------------------------
# Gráfico de resultados
# ------------------------------------------------------------------
fig, ax = plt.subplots(figsize=(8, 4))

# paleta de colores gris = histórico || azul = punto máximo
clrs = [_Gray_ if (x < max(resultados_metricas['Porcentaje (%)'])) else _Light_Blue_ for x in resultados_metricas['Porcentaje (%)']]

# Crear gráfico de barras
ax = sns.barplot(
    x='Porcentaje (%)',
    y='Métrica',
    data=resultados_metricas,
    palette=clrs
)

# etiquetas de datos en las barras
for p in ax.patches:
    width = p.get_width()
    plt.text(
        width + 1,       # Posición X (un poco a la derecha de la barra)
        p.get_y() + p.get_height() / 2, # Posición Y (centro de la barra)
        f'{width:,.0f}%', # Valor formateado
```

```python
        ha='left',
        va='center',
        fontweight='bold',
        fontsize=10,
        color=_color_gray
    )


# titulo narrativo
plt.title("KPIs de Calidad de Datos - Dataset Integrado",
          fontsize=16, fontweight='bold', loc='left', pad=20)

# subtitulo y contexto
ax.text(x=-0.5, y=ax.get_ylim()[1] * 1.02,
        s="Nivel de cumplimiento de los datos.",
        fontsize=11, color=_color_gray, ha='left')


# formato del gráfico
plt.xlim(0, 110) # Límite hasta 110 para dar espacio a las etiquetas
plt.ylabel('', fontsize=12)

# limpieza de ruido y eliminacion de bordes
ax.spines['top'].set_visible(False)
ax.spines['bottom'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.get_yaxis().set_visible(True)
ax.get_xaxis().set_visible(False)

# guardar imagen y mostrar el gráfico
plt.tight_layout()
plt.savefig('kpis_calidad_datos.png', dpi=300)
plt.show()

print("\n✅ Visualización generada y guardada como 'kpis_calidad_datos.png'")
```




```
✅ Visualización generada y guardada como 'kpis_calidad_datos.png'
```

📊 Comparación de calidad antes y después

A continuación se realiza una comparación de calidad "antes vs. después" sobre el dataset integrado usando un flujo de limpieza ligero y métricas de completitud, consistencia y precisión aritmética. Mediante esta comparación se podrá demostrar el valor del trabajo técnico realizado hasta ahorita.

⚙️ Script de comparación de calidad antes vs después

In [33]:

```python
# ----------------------------------------------------------------
# ⚙️ Script de comparación de calidad antes vs después
# ----------------------------------------------------------------

display_mensaje("Comparación de calidad antes y después de la integración")

# dataset de ventas original
df_sales = pd.read_csv('product_sales_cleaned.csv')
# catalogo de productos original
df_fashion_raw = pd.read_csv('fashion_products_integrated.csv')

# cruce básico de informacion de los productos
```

```python
df_sales['product_base'] = df_sales['product_name'].apply(lambda x: str(x).split()[-1].capitalize())
df_fashion_raw['product_base'] = df_fashion_raw['product_name'].str.capitalize()

# dataset sucio antes de la integracion
df_before = pd.merge(
    df_sales,
    df_fashion_raw[['product_base', 'brand', 'rating', 'color', 'size']],
    on='product_base',
    how='left'
)

# ------------------------------------------------------------------
# dataset integrado y limpio
df_after = pd.read_csv('master_sales_integrated.csv')

# calculo de metricas de evaluacion
def calcular_metricas(df, is_before=True):
    total_rows = len(df)

    # 1. Completitud en todas las filas
    total_cells = total_rows * df.shape[1]
    missing_cells = df.isnull().sum().sum()
    completitud = ((total_cells - missing_cells) / total_cells) * 100

    # 2. Consistencia
    col_qty = 'quantity'
    col_price = '_unit_price_'
    col_rev = '_revenue_'

    # evaluacion de consistencia
    try:
        calculated_rev = df[col_qty] * df[col_price]
        consistencia = ((abs(df[col_rev] - calculated_rev) < 0.01).sum() / total_rows) * 100
    except:
        consistencia = 0.0

    # 3. Precisión en rating
    col_rating = 'rating'
    try:
        # Los nulos en Rating afectarán la precisión en el "Antes"
        precisos = ((df[col_rating] >= 1.0) & (df[col_rating] <= 5.0)).sum()
        precision = (precisos / total_rows) * 100
    except:
        precision = 0.0

    return completitud, consistencia, precision

# ------------------------------------------------------------------
```

```python
# Ejecución de calculos
# ------------------------------------------------------------------
comp_b, cons_b, prec_b = calcular_metricas(df_before, is_before=True)
comp_a, cons_a, prec_a = calcular_metricas(df_after, is_before=False)

# ------------------------------------------------------------------
# Preparacion de datos para visualizar
# ------------------------------------------------------------------
data_vizualizar = {
    'Métrica': ['Completitud', 'Completitud', 'Consistencia', 'Consistencia', 'Precisión', 'Precisión'],
    'Estado': ['Antes', 'Después', 'Antes', 'Después', 'Antes', 'Después'],
    'Porcentaje (%)': [comp_b, comp_a, cons_b, cons_a, prec_b, prec_a]
}
df_viz = pd.DataFrame(data_vizualizar)

print("\n📊 Resultados Comparativos:")
display(df_viz.pivot(index='Métrica', columns='Estado', values='Porcentaje (%)').round(2))
```

```
================================================================================
 --- Comparación de calidad antes y después de la integración ---
================================================================================

📊 Resultados Comparativos:
```

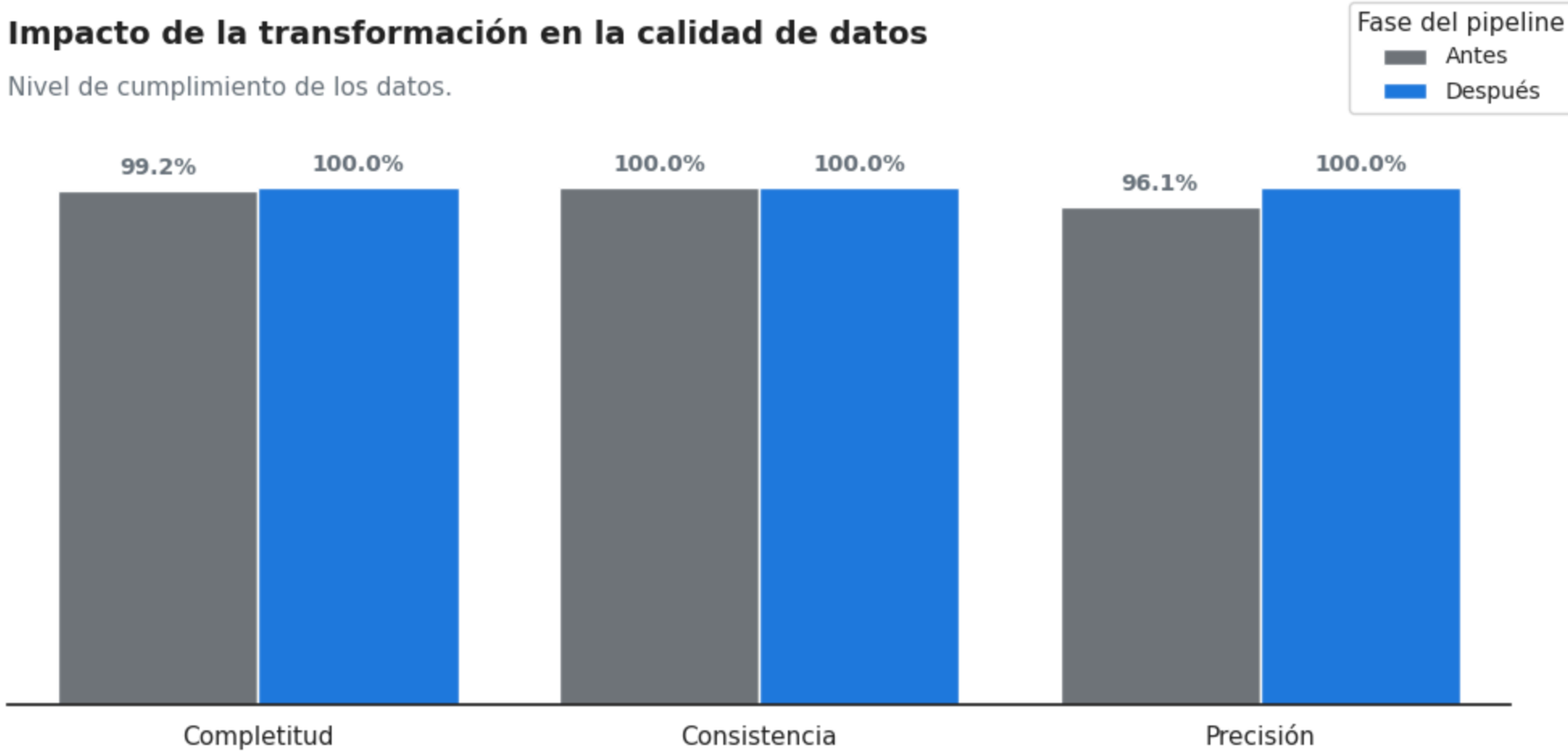
Estado	Antes	Después
Métrica		
Completitud	99.21	100.0
Consistencia	100.00	100.0
Precisión	96.05	100.0

📊 Gráfica de comparación

In [55]:

```python
# ------------------------------------------------------------------
# Gráfica de comparación
# ------------------------------------------------------------------
fig, ax = plt.subplots(figsize=(10, 5))

# grafica de barras
ax = sns.barplot(
    data=df_viz,
    x='Métrica',
    y='Porcentaje (%)',
    hue='Estado',
```

```python
    palette=[_color_gray, _color_primary]
)

# etiquetas de valor a las barras
for p in ax.patches:
    height = p.get_height()
    if height > 0:
        ax.annotate(f'{height:.1f}%',
                    (p.get_x() + p.get_width() / 2., height),
                    ha = 'center', va = 'bottom',
                    fontsize=10, fontweight='bold', color=_color_gray,
                    xytext = (0, 5), textcoords = 'offset points')


# titulo narrativo
plt.title("Impacto de la transformación en la calidad de datos",
          fontsize=14, fontweight='bold', loc='left')

# subtitulo y contexto
ax.text(x=-0.5, y=ax.get_ylim()[1] * 1.12,
        s="Nivel de cumplimiento de los datos.",
        fontsize=11, color=_color_gray, ha='left')


# formato del gráfico
plt.ylim(0, 125) # Espacio extra para las etiquetas
plt.ylabel('', fontsize=12)
plt.xlabel('', fontsize=12)
plt.legend(title='Fase del pipeline', bbox_to_anchor=(1.05, 1), loc='center right', fontsize=10, title_fontsize=11)

# limpieza de ruido y eliminacion de bordes
ax.spines['top'].set_visible(False)
ax.spines['bottom'].set_visible(True)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.get_yaxis().set_visible(False)
ax.get_xaxis().set_visible(True)

# guardar imagen y mostrar el gráfico
plt.tight_layout()
plt.savefig('comparativa_calidad_datos.png', dpi=300)
plt.show()

print("\n✅ Visualización comparativa generada y guardada como 'comparativa_calidad_datos.png'")
```




```
✅ Visualización comparativa generada y guardada como 'comparativa_calidad_datos.png'
```

👨‍⚖️ Incorporación de principios normativos y de gobernanza

Dado el material de estudios y Gobierno de Datos un pipeline automatizado carece de valor empresarial si no cumple con las regulaciones de protección y calidad de la información, es por ello que se presenta a continuación la documentación de como el pipeline se alinea con la Norma ISO/IEC 25012, el marco DAMA-DMBOK y la legislación de privacidad GDPR & LFPDPPP.

⬛ Marco normativo y de gobernanza del pipeline

El ecosistema de datos diseñado en esta evidencia final, que consta de una fuente de datos sobre registros de ventas y atributos añadidos, se sustenta en tres pilares normativos fundamentales, los cuales se han integrado en el código del pipeline.

1️⃣ DAMA-DMBOK

> El marco de la Data Management Association establece que la calidad de los datos depende directamente de entender el linaje y origen de los mismos.

La creación de reportes HTML mediante Great Expectations es la automatización de la *Gestión de metadatos* que exige DAMA, y, al registrar cada *ValidationDefinition* sobre la suite de datos, se ha establecido una auditoría de alto nivel sobre el dataset, con lo cuál, se podrá corroborar bajo qué reglas y en qué momento la fuente de datos fue evaluada.

2️⃣ ISO/IEC 25012 y 8000

> La ISO 8000 aborda la calidad de los datos a nivel corporativo.
>
> Mientras que la 25012 se enfoca en el producto de software/datos.

Para cumplir con el principio de exactitud y credibilidad de la ISO 25012 se diseño la función de *extracción personalizada de marcas* y la *consolidación de productos mediante agrupaciones* antes del Merge, esto en la integración de las fuentes de datos. De esta manera se previene la distorsión de la variable Revenue y se evitan registros duplicados.

Para cumplir con el principio de accesibilidad de la ISO 8000 se implemento una estandarización de encabezados en los dataset ademas de asegurar el casting de tipos en todas las variables, lo que asegura el intercambio de datos de forma segura y automática.

3️⃣ GDPR / LFPDPPP

Dentro del pipeline, al forzar el casting de `customer_id` y `product_id` a tipo *string* se estableció una política técnica de pseudonimización.

Por otro lado, la métrica de *Completitud* y la prevención de *Nulos* en variables demográficas o descriptivas aseguran que el modelo predictivo no sufra de "Sesgo por Omisión", garantizando decisiones algorítmicas más justas.

Sin embargo, las variables de datos del cliente como `customer_id` y `customer_name` siguen estando presentes en el dataset integrado, y cruzan los limites del procesamiento de Datos Personales, lo que se debería marcar como prioridad para una segunda actualización del pipeline donde se puedan anonimizar estos datos personales.

🤖 Automatización del monitoreo de calidad

En este paso se transforma el pipeline de un simple análisis en un activo para analizar datos de manera automática. Esto es muy importante debido a que en la vida real, los datos cambian todos los días; por lo tanto, la calidad no puede ser una revisión manual de una sola vez, sino un proceso de monitoreo automatizado y alertamiento.

A continuación, se implementa un script en Python que simula un proceso de revisión automática y registro de eventos, el script evalúa diferentes métricas y generará una alerta si los niveles caen por debajo del umbral permitido. En la vida real, este script podría ejecutarse como un "Job" programado.

🚚 Estrategia para el monitoreo automatizado

- Métricas a monitorear de manera automática:
  - Completitud: % de celdas no nulas en todo el dataset
  - Unicidad: % de Order_IDs únicos
  - Consistencia: % de filas donde Revenue == Quantity * *unit_price* (considerando un margen de error por decimales)
  - Precisión: % de filas con Rating válido y Precios válidos
  - Puntualidad: % de fechas válidas
- Umbral de tolerancia: se establecerá un 98.0% como estándar para todas las métricas. Si la consistencia es menor, el sistema lo entenderá como un incidente de calidad. *Podria ocurrir que diferentes métricas tengan diferentes umbrales de tolerancia, esto mismo se define en el script.*
- Acciones automatizadas: El script tendrá la capacidad de escribir registros en una bitácora de auditoría y disparar alertas para cuando las métricas no sean exitosas.

⚙️ Script de automatización

In [9]:
```
# ---------------------------------------------------------------------------
# ⚙️ Script de automatización
# ---------------------------------------------------------------------------

import pandas as pd
import logging
from datetime import datetime

# ---------------------------------------------------------------------------
# configuración del registro de eventos
```

```python
# ---------------------------------------------------------------------------
'''
# se crea un archivo físico que guarda el historial de monitoreo,
# cumpliendo normas ISO 27001 y auditorías internas.
'''
logging.basicConfig(
    filename='monitoreo_calidad_datos.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

# ---------------------------------------------------------------------------
# definición de umbrales de tolerancia
# nota: se deja a la metrica Puntualidad con una tolerancia minima del 110% como caso de estudio
# ---------------------------------------------------------------------------
_UMBRALES_METRICAS_ = pd.DataFrame({
    'metrica': ['Completitud', 'Unicidad', 'Consistencia', 'Precisión', 'Puntualidad'],
    'umbral_minimo': [98.0, 98.0, 98.0, 98.0, 110]
})

# ---------------------------------------------------------------------------
# envio de alertas automaticas
# ---------------------------------------------------------------------------
def enviar_alerta_equipo_datos(mensaje):
    print("\n" + "="*60)
    print("🔔 [SISTEMA DE ALERTAS AUTOMATIZADO]")
    print(f"🚀 Enviando notificación webhook al canal #alertas-calidad-datos...")
    print(f"📩 Payload transmitido: {{'Severidad': 'Alta', 'Mensaje': '{mensaje}'}}")
    print("="*60 + "\n")


# ---------------------------------------------------------------------------
# evaluacion de métricas de calidad
# ---------------------------------------------------------------------------
def evaluar_metricas_calidad(ruta_archivo):

    display_mensaje("CALCULANDO MÉTRICAS DE CALIDAD DE DATOS")

    # dataset integrado
    df_metricas = pd.read_csv(ruta_archivo)
    df_metricas['order_date'] = pd.to_datetime(df_metricas['order_date'])

    total_rows = len(df_metricas)

    # ---------------------------------------------------------------------------
    # calculando métricas de calidad

    # 1. Completitud: % de celdas no nulas en todo el dataset
```

```python
    total_cells = total_rows * df_metricas.shape[1]
    missing_cells = df_metricas.isnull().sum().sum()
    _COMPLETITUD_ = ((total_cells - missing_cells) / total_cells) * 100

    # 2. Unicidad: % de Order_IDs únicos
    _UNICICIDAD_ = (df_metricas['order_id'].nunique() / total_rows) * 100

    # 3. Consistencia: % de filas donde Revenue == Quantity * _unit_price_ (considerando un margen de error por decimales)
    df_metricas['calculated_revenue'] = df_metricas['quantity'] * df_metricas['_unit_price_']
    filas_consistentes = (abs(df_metricas['_revenue_'] - df_metricas['calculated_revenue']) < 0.01).sum()
    _CONSISTENCIA_ = (filas_consistentes / total_rows) * 100

    # 4. Precisión: % de filas con Rating válido y Precios válidos
    filas_precisas = ((df_metricas['rating'] >= 1.0) & (df_metricas['rating'] <= 5.0) & (df_metricas['_unit_price_'] > 0)).sum()
    _PRECISION_ = (filas_precisas / total_rows) * 100

    # 5. Puntualidad: % de fechas válidas
    # se define la fecha de corte como el 31 de diciembre de 2024
    fecha_corte = pd.to_datetime('2024-12-31')
    filas_puntuales = (df_metricas['order_date'] <= fecha_corte).sum()
    _PUNTUALIDAD_ = (filas_puntuales / total_rows) * 100

    # ---------------------------------------------------------------------------
    # resultados de las metricas para reportes
    resultados_metricas = pd.DataFrame({
        'Métrica': ['Completitud', 'Unicidad', 'Consistencia', 'Precisión', 'Puntualidad'],
        'Porcentaje (%)': [_COMPLETITUD_, _UNICICIDAD_, _CONSISTENCIA_, _PRECISION_, _PUNTUALIDAD_]
    })

    return resultados_metricas


# ---------------------------------------------------------------------------
# tarea automatica de monitoreo
# ---------------------------------------------------------------------------
def ejecutar_job_monitoreo(ruta_archivo, umbrales_metricas):
    timestamp_actual = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    print(f"--- INICIANDO EJECUCIÓN DE MONITOREO ({timestamp_actual}) ---")

    try:
        # evaluación de métricas de calidad
        resultados_metricas = evaluar_metricas_calidad(ruta_archivo)

        # evaluación de resultados en las metricas
        for index, row in resultados_metricas.iterrows():
            logging.info(f"Métrica: {row['Métrica']}, Valor: {row['Porcentaje (%)']:.2f}%")
            metrica_name = row['Métrica']
            metrica_value = row['Porcentaje (%)']
```

```python
            # obteniendo el umbral mínimo específico para la métrica actual desde el DataFrame de umbrales
            umbral_minimo = umbrales_metricas.loc[umbrales_metricas['metrica'] == metrica_name, 'umbral_minimo'].values[0]

            # ------------------------------------------------------------------------
            # caso de falla
            if metrica_value < umbral_minimo:
                # En caso de falla: alertas y registrar error crítico
                msg_error = f"ALERTA DE CALIDAD: La métrica de calidad: {metrica_name} con valor: ({metrica_value:.2f}%) es menor al umbral exigido ({umbral_minimo}%)."
                logging.error(msg_error)
                print(f"❌ FALLO DE CONTROL: {msg_error}")

                # protocolo de notificación
                enviar_alerta_equipo_datos(msg_error)

            # ------------------------------------------------------------------------
            # caso de exito
            else:
                # En caso de exito: se registra en bitácora
                msg_ok = f"Control superado. Metrica: {metrica_name} con valor: {metrica_value:.2f}%. Umbral: {umbral_minimo}%."
                logging.info(msg_ok)
                print(f"✅ ESTADO SALUDABLE: {msg_ok}")

    except FileNotFoundError:
        msg_archivo = f"Error crítico: No se encontró el archivo '{ruta_archivo}' para el monitoreo."
        logging.critical(msg_archivo)
        print(f"⚠️ {msg_archivo}")
    except Exception as e:
        msg_excepcion = f"Fallo inesperado en el pipeline de monitoreo: {str(e)}"
        logging.critical(msg_excepcion)
        print(f"⚠️ {msg_excepcion}")

# ----------------------------------------------------------------------------
# Ejecución de la tarea automatica
# ----------------------------------------------------------------------------
ejecutar_job_monitoreo('master_sales_integrated.csv', _UMBRALES_METRICAS_)
```

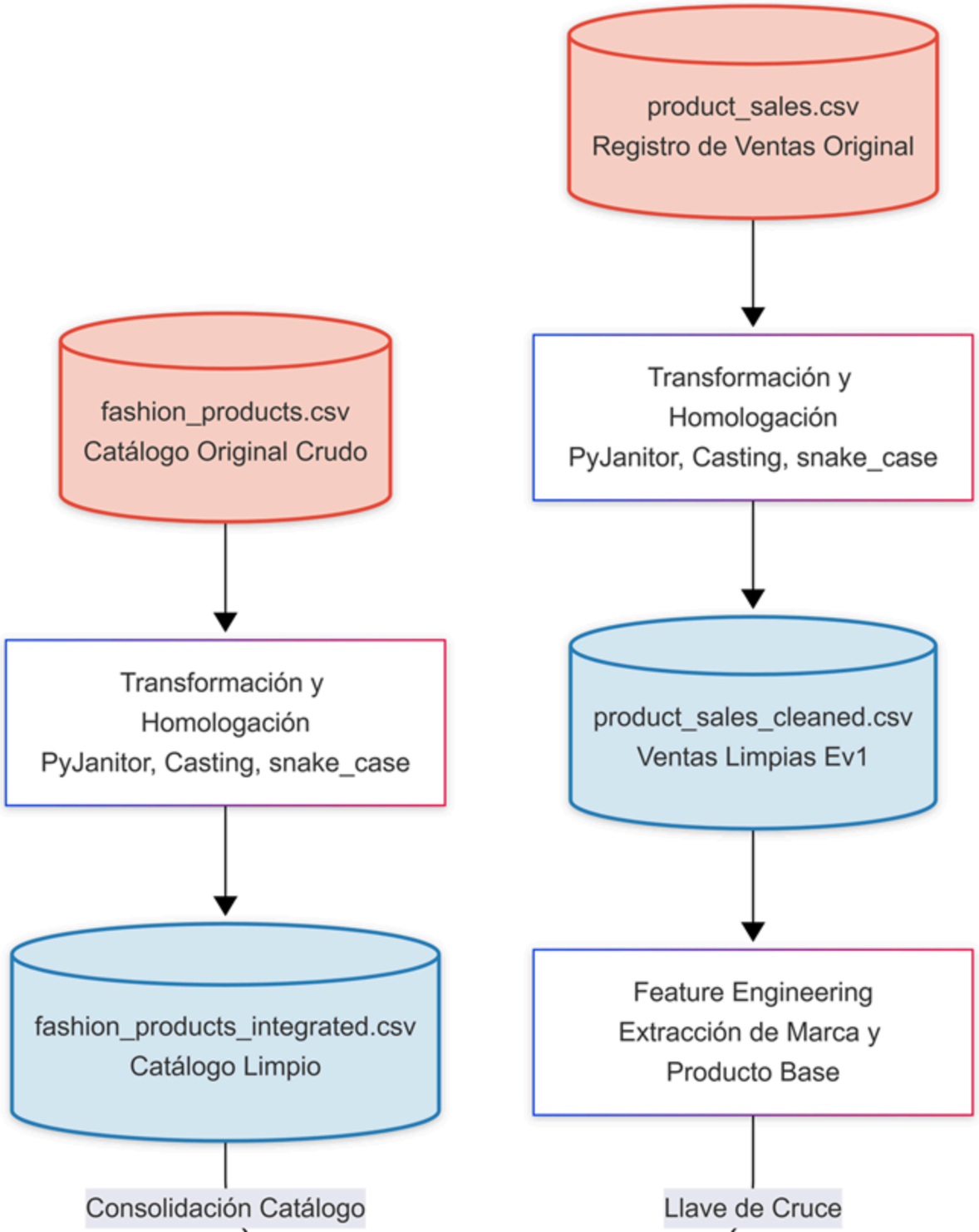
```
--- INICIANDO EJECUCIÓN DE MONITOREO (2026-03-11 14:11:40) ---

================================================================================
 --- CALCULANDO MÉTRICAS DE CALIDAD DE DATOS ---
================================================================================
✅ ESTADO SALUDABLE: Control superado. Metrica: Completitud con valor: 100.00%. Umbral: 98.0%.
✅ ESTADO SALUDABLE: Control superado. Metrica: Unicidad con valor: 100.00%. Umbral: 98.0%.
✅ ESTADO SALUDABLE: Control superado. Metrica: Consistencia con valor: 100.00%. Umbral: 98.0%.
✅ ESTADO SALUDABLE: Control superado. Metrica: Precisión con valor: 100.00%. Umbral: 98.0%.
❌ FALLO DE CONTROL: ALERTA DE CALIDAD: La métrica de calidad: Puntualidad con valor: (100.00%) es menor al umbral exigido (110.0%).

============================================================
🔔 [SISTEMA DE ALERTAS AUTOMATIZADO]
🚀 Enviando notificación webhook al canal #alertas-calidad-datos...
📩 Payload transmitido: {'Severidad': 'Alta', 'Mensaje': 'ALERTA DE CALIDAD: La métrica de calidad: Puntualidad con valor: (100.00%) es menor al umbral exigido (110.0%).'}
============================================================
```

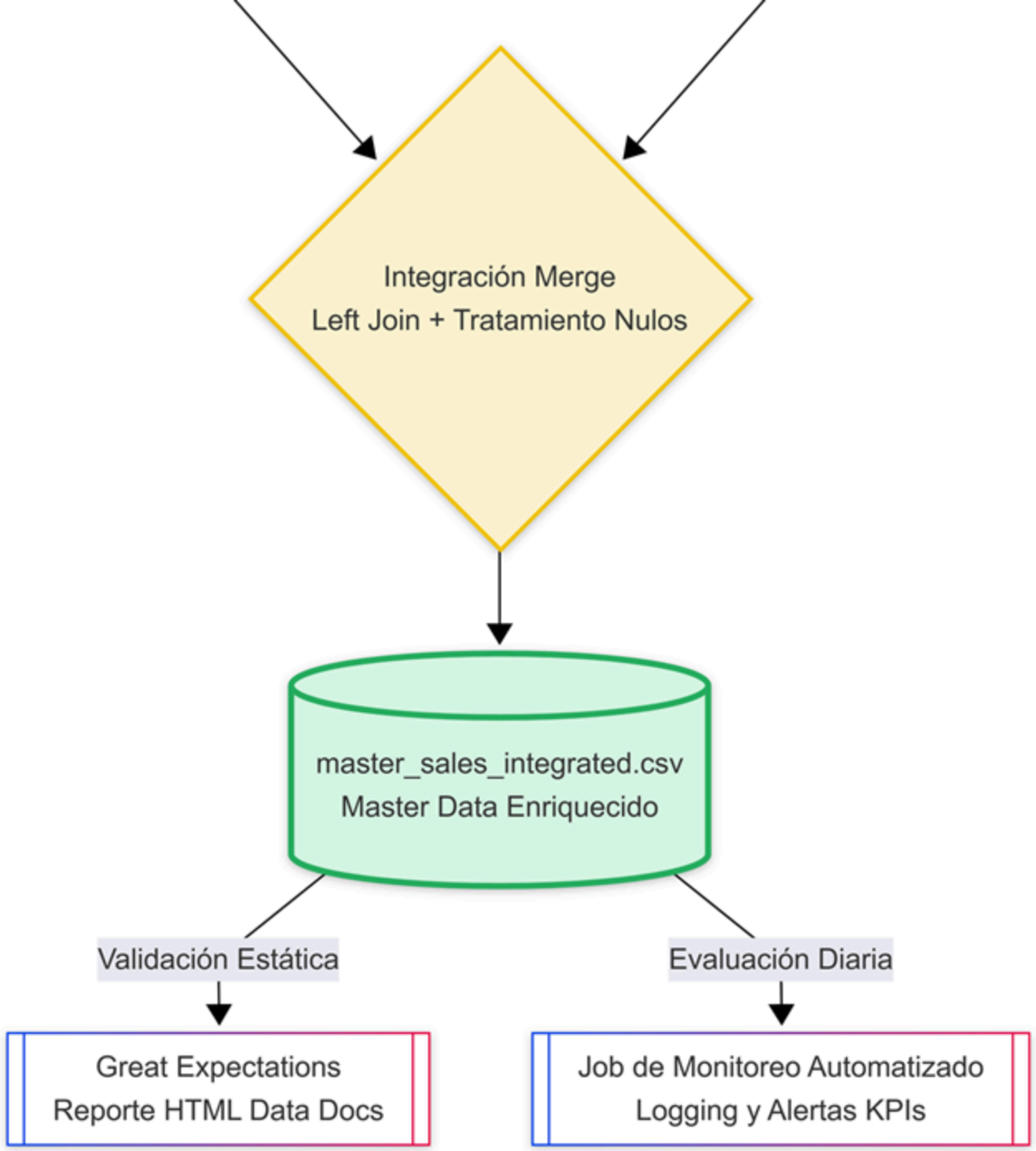
El script presentado no limpia datos, solo verifica los procesos previos de limpieza e integración, evaluando el nivel de las métricas junto con sus correspondientes umbrales de tolerancia, permitiendo que el monitoreo sea ligero y rápido con el único objetivo de evaluar de manera automática la calidad final del dataset integrado.

La creación del archivo `monitoreo_calidad_datos.log` permite tener un historial de la salud de la fuente de datos integrada cada vez que pasa por el sistema.

Adicional, en la función `enviar_alerta_equipo_datos`, se podrían usar librerías adicional para conectar este script con API's externas para crear notificaciones y/o enviar emails.

⤵️ Diagrama general del flujo de datos


product_sales.csv
Registro de Ventas Original
fashion_products.csv
Catálogo Original Crudo
Transformación y
Homologación
PyJanitor, Casting, snake_case
Transformación y
Homologación
PyJanitor, Casting, snake_case
product_sales_cleaned.csv
Ventas Limpias Ev1
fashion_products_integrated.csv
Catálogo Limpio
Feature Engineering
Extracción de Marca y
Producto Base
Consolidación Catálogo
Llave de Cruce


Integración Merge
Left Join + Tratamiento Nulos
master_sales_integrated.csv
Master Data Enriquecido
Validación Estática
Evaluación Diaria
Great Expectations
Reporte HTML Data Docs
Job de Monitoreo Automatizado
Logging y Alertas KPIs

🍃 Reflexión crítica

⚖️ Importancia del gobierno de datos y el cumplimiento normativo

En cualquier sistema moderno, la tarea de procesar datos de forma masiva garantiza la confiabilidad, seguridad y ética de los datos. De esta manera, el **Gobierno de Datos** actúa como el puente entre la tecnología y el negocio, asegurando que los activos de información estén alineados con los objetivos corporativos, y que además, cumplan con estándares internacionales de calidad.

Asimismo, el cumplimiento normativo como la LFPDPPP se ha convertido en una parte fundamental para asegurar la confianza del cliente, pues hoy en día es sumamente importante que los pipelines automatizados contemplen la privacidad de los datos desde el diseño y antes de escribir la primer linea de código. Es así como la Gobernanza de datos se debe percibir como esa última instancia que se encarga de responsabilizarse del impacto algorítmico y humano que tendrán las decisiones basadas en los datos.

☣️ Riesgos, decisiones y mejoras logradas en el pipeline

Riesgos mitigados

- Se mitigó el riesgo de que datos corruptos llegaran a los modelos analíticos sin ser detectados, además, la implementación de alertas automatizadas basadas en la evaluación de métricas vs umbrales de tolerancia, asegura que el equipo sea notificado antes de que el error impacte en el negocio.
- Durante la integración de los datasets se realizaron transformaciones a los identificadores `customer_id` y `product_id` con lo que se previnieron agrupaciones o cálculos matemáticos accidentales.
- Asimismo, durante el proceso de integración se ejecuto una validación de rangos y dominios en variables como `rating` y `talla` previniendo así el entrenamiento de modelos predictivos con datos anómalos o irreales.

Decisiones tomadas

- Para lograr un merge exitoso entre las distintas fuentes de tatos se decidió extraer la raíz del producto y la marca directamente del texto libre de las ventas.
- Y se optó por agrupar y promediar el catálogo de moda antes del Merge, protegiendo la integridad de los 200,000 registros originales.
- Por otro lado, en el script de monitoreo automatizado, los umbrales de calidad se parametrizaron en un DataFrame independiente al script de evaluación, lo que permite poder ajustar los umbrales sin la necesidad de modificar el código de evaluación.

Mejoras logradas

- El dataset original fue mejorado con contexto de catálogo, añadiendo las columnas Marca, Color, Talla y Rating, elevando de esta manera su valor dentro del negocio para la extracción de insights comerciales.
- Se pasó de tener solo un reporte técnico HTML mediante Great Expectations, a la generación de KPIs de calidad, dando la posibilidad de auditar dimensiones de Unicidad, Puntualidad y Precisión de manera rápida y sencilla.

- Con el registro de eventos en el script automatizado, el pipeline ahora puede generar una bitácora histórica y auditable, con lo cual, además de ser una herramienta de ayuda para auditorias internas, se cumple aún más con la ISO 27001.